

Dokumentation der Flugbahn-Zuordnung

Dieses Kapitel beschreibt die Funktionsweise des Moduls `trajectory.py`. Das Modul dient der Ermittlung von Flugbahnen (Trajektorien) aus einer Reihe von Zeitstempeln und ENU-Koordinaten. Dabei berechnet das Modul durch Bewegungsvorhersagen und Distanzprüfungen zusammenhängende Tracks und weist jeder Beobachtung eine `determined_track_id` zu.

Kernfunktion: Trajektorien-Ermittlung (`determine_tracks`)

Die Hauptfunktion `determine_tracks` bildet das Herzstück des Moduls. Sie nimmt eine Liste von Rohdaten (Records) entgegen und gibt diese mit der neu berechneten Track-ID zurück.

1. Initialisierung und Startbedingungen

Vor der eigentlichen Iteration ist die Liste der aktiven Tracks (`active_tracks`) **leer**. Es existieren noch keine Flugbahnen. Der Zähler für die nächste Track-ID (`next_track_id`) startet bei 1. Die Beobachtungen werden zunächst chronologisch sortiert und in einem Dictionary nach exakten Zeitstempeln gruppiert.

2. Iteration über die Zeitstempel

Die Funktion durchläuft alle Zeitstempel in aufsteigender Reihenfolge. Für jeden Zeitschritt passieren folgende Aktionen:

A. Track-Bereinigung:

Beim allerersten Zeitstempel entfällt dieser Schritt, da noch keine Tracks vorhanden sind. Bei allen folgenden Zeitstempeln werden zuerst alle Tracks aus `active_tracks` gelöscht, deren letzte Beobachtung länger als `MAX_GAP_SECONDS` zurückliegt. Sie gelten als abgeschlossen.

B. Vorhersage und Kandidatensuche:

Um zu entscheiden, ob eine aktuelle Beobachtung zu einem bestehenden Track gehört, wird für jeden aktiven Track ermittelt, wo sich das Objekt nach der vergangenen Zeit laut seiner letzten Geschwindigkeit gerade befinden *müsste*. Daraus ergibt sich ein vorhergesagter Punkt im 3D-Raum.

Anschließend wird für jede aktuelle Beobachtung die tatsächliche räumliche Distanz zu diesem vorhergesagten Punkt berechnet.

Ein möglicher Zusammenhang (Kandidat) gilt als plausibel, wenn diese Distanz kleiner ist als die theoretisch maximal zurücklegbare Strecke. Diese ergibt sich aus der Formel:

Maximaldistanz = (maximale Geschwindigkeit × vergangene Sekunden) + Toleranzpuffer
(`DISTANCE_MARGIN`).

Ist die berechnete Distanz kleiner als dieser Grenzwert, wird das Paar (bestehend aus Track und Beobachtung) samt Distanz als Kandidat für die Zuordnung gespeichert.

C. Greedy-Zuordnung (Zuweisung anhand der kürzesten Distanz):

Da eine Beobachtung möglicherweise mehreren Tracks als Kandidat dient oder ein Track mehrere nahe Beobachtungen hat, muss eine eindeutige Zuordnung erfolgen. Dies geschieht nach dem Greedy-Prinzip (vom kleinsten Abstand zum größten) in folgenden Schritten:

1. Alle in Schritt B gefundenen Kandidaten werden nach ihrer berechneten Distanz aufsteigend sortiert (der Kandidat mit der geringsten Distanz steht ganz vorne).
2. Die Liste wird von oben nach unten abgearbeitet.
3. Für jeden Kandidaten wird geprüft: Wurde dieser Track in diesem Zeitschritt *bereits* einer anderen Beobachtung zugeordnet? Oder wurde diese Beobachtung *bereits* einem anderen Track zugeordnet? Wenn ja, wird dieser Kandidat ignoriert.
4. Wenn beide noch "frei" sind, wird die Beobachtung fest dem Track zugeordnet. Der Track wird intern mit den neuen Koordinaten und der neuen Geschwindigkeit aktualisiert. Der Track und die Beobachtung werden für den restlichen Verlauf dieses Zeitschritts als "vergeben" markiert.
5. Dieser Vorgang wird fortgesetzt, bis die Liste abgearbeitet ist. Dadurch ist sichergestellt, dass immer die wahrscheinlichsten (nächsten) Paarungen zuerst vergeben werden.

D. Erstellung neuer Tracks:

Da beim ersten Zeitstempel keine aktiven Tracks existieren, springt die Funktion direkt in diesen Schritt: Alle Beobachtungen des ersten Zeitstempels gründen jeweils einen neuen Track.

Bei allen folgenden Zeitstempeln betrifft dies nur die Beobachtungen, die in Schritt C keinem bestehenden Track zugeordnet werden konnten. Jede dieser übrigen Beobachtungen startet einen neuen Track mit der nächsten verfügbaren ID und wird zu `active_tracks` hinzugefügt.

3. Datenrückgabe

Zum Abschluss der gesamten Iteration wird jeder ursprünglichen Beobachtung die ermittelte Track-ID im Format `track_XXXX` zugewiesen und die Datensätze zurückgegeben.

Hilfsfunktionen und Datenstrukturen

`parse_timestamp`

Wandelt einen ISO-8601-Zeitstempel-String in ein `datetime` -Objekt um, wobei eventuelle "Z"-Suffixe korrekt in Zeitzone-Offsets umgewandelt werden.

euclidean_distance

Berechnet die euklidische Distanz zwischen zwei 3D-Punkten (X, Y, Z).

Observation und Track (Dataclasses)

Diese Datenklassen kapseln die Zustände:

- **Observation** : Speichert Index, Zeitstempel, ENU-Koordinaten und den Rohdaten-Dictionary.
- **Track** : Verwaltet die Track-ID sowie die letzte Position und Geschwindigkeit. Enthält zwei zentrale Methoden:
 - `predict_position` : Berechnet die voraussichtliche Position anhand der vergangenen Zeit und der aktuellen Geschwindigkeit.
 - `update` : Aktualisiert den Track mit einer neuen Beobachtung. Die neue Geschwindigkeit wird mittels Glättungsfaktor (`velocity_smoothing` , Standard 0.7) aus der gemessenen und der vorherigen Geschwindigkeit berechnet.

load_observations

Liest die Rohdaten aus, extrahiert die Felder `timestamp` , `enu_e` , `enu_n` und `enu_u` und erstellt daraus die sortierte Liste der `Observation` -Objekte.

Parameter und Ausführung (main)

Die `main` -Funktion lädt die Daten aus einer Eingabedatei, ruft `determine_tracks` auf und speichert das Ergebnis.

Parameter	Standardwert	Beschreibung
<code>INPUT_FILE</code>	<code>vogel_flugbahnen.json</code>	Pfad zur Eingabedatei
<code>OUTPUT_FILE</code>	<code>vogel_flugbahnen_determined.json</code>	Pfad zur Ausgabedatei
<code>MAX_GAP_SECONDS</code>	10	Maximale Zeitlücke in Sekunden, nach der ein Track geschlossen wird
<code>MAX_SPEED_UNITS_PER_SEC</code>	25	Maximal plausible Geschwindigkeit pro Sekunde
<code>DISTANCE_MARGIN</code>	20	Zusätzlicher Toleranzpuffer für

Parameter	Standardwert	Beschreibung
		die Distanzberechnung